

Minor Exam

● Graded

Student

Sarthak Panda

Total Points

38.5 / 60 pts

Question 1

Question 1

5 / 10 pts

1.1

Part a

2 / 2 pts

✓ + 1 pt No

✓ + 1 pt Timer Interrupt

+ 0.5 pts Scheduler or preemptive multi-tasking

+ 0 pts Incorrect

1.2

Part b

0 / 4 pts

+ 2 pts Alarm based system

+ 1 pt Advantages of this scheme

+ 1 pt Explanation of the scheme

✓ + 0 pts Incorrect

1.3

Part c

3 / 4 pts

✓ + 1 pt **Inter-Processor Interrupts** : Identify the use of IPIs as the synchronization

✓ + 1 pt **Master Core Role**: how the core with timer access serves as a coordinator

✓ + 1 pt **Context Switch Triggering** :how IPIs trigger context switches on cores without timer access

+ 1 pt **Scheduling Process**: how the system maintains fair scheduling across all cores

+ 0.5 pts Fair scheduling

+ 0 pts Incorrect

Question 2

Question 2

2.5 / 10 pts

2.1 Part a

2 / 3 pts

✓ + 1 pt Load both the DLLs (new & old) in **memory** rather than patching/overwriting existing DLL in-memory.

+ 1 pt Utilize concept of **stub function/dispatch table/indirection table** to identify which DLL's function is to be called.

✓ + 1 pt Use of **function pointers** in the described concept

+ 0 pts *Incorrect*

1 internally, stub function would allow the kernel to identify which DLL's function call is it be made

2.2 Part b

0.5 / 2 pts

+ 0.5 pts Use **reference counter** to count Thread_ids using the specific version of DLL, to indicate which threads are using.

+ 1 pt Explanation for **counter value** = 0 -> i.e., remove the old DLL.

+ 0.5 pts Method for updation of the counter value in the stub function.

+ 0 pts *Incorrect*

💬 + 0.5 pts Used pointer concept to update mappings in memory for new, old DLLs. (however, no mention of stub functions, or how it'd be performed)

2.3 Part c

0 / 2 pts

+ 2 pts Resource information is stored in the task_struct of a process. file_struct for open files, mm_struct for memory mapping and socket handles for n/w connections. If the concept of transferring from task_struct is explained (i.e., no need to do any transfer, just pass the pointers, because new DLL and old DLL will share the same task_struct.)

✓ + 0 pts incorrect

+ 1 pt If some components of the complete rubric are missing. Please tally your answer with the provided rubric to see what is missing.

- + 1.5 pts Concept of dispatch table/indirection table/stub functions is explained
- + 1.5 pts concept of keeping the old and new DLL function entries in the dispatch/indirection table is explained
- + 3 pts Concept of stub function/wrapper function, need to keep old and new DLL function pointer in stub explained and function adapters(functions that can change the input signatures based on the context of the request and direct to the right DLL function explained)
- + 3 pts Concept of Dynamic Binary patching explained appropriately

✓ + 0 pts incorrect

- + 1 pt concept of keeping old DLL and new DLL alive is explained
- + 1.5 pts Partially explained dynamic binary patching
- + 0 pts Click here to replace this description.

Question 3

Question 3

8 / 10 pts

3.1 Part a

3 / 3 pts

✓ + 1 pt Yes, Share all memory regions except stack and TLS

✓ + 2 pts Justification: Fast inter-thread communication, easy **context switch** between threads

+ 0 pts Not correct

3.2 Part b

3 / 3 pts

✓ + 3 pts Each thread has its own private stack. These stacks are arranged in the shared virtual memory space one after the other such that they **do not overlap or interfere** with each other.

+ 0 pts Not correct

3.3 Part c

2 / 4 pts

+ 1 pt Segment registers should be in the context of a thread. During thread context switch, the OS will load stack segment selector of newly scheduled thread into the segment register.

✓ + 2 pts Each thread's stack segment has a base (start of the stack) and a limit (max size). For a thread, Base + offset should lie in the segment (otherwise segmentation fault).

+ 1 pt Thread should not overflow its heap allocation as it can overwrite another thread's stack, basically entire space needs to be protected.

+ 0 pts Not correct

Question 4

Question 4

10 / 10 pts

4.1 Part a

3 / 3 pts

+ 1.5 pts Binary trees have a large depth, requiring multiple nodes along the search path to be locked during delete or update operations. This increases contention and limits parallelism, reducing system efficiency.

*Locking entire path from root to target node lock more nodes

*Implementing the only parent node and this node lock system is complex.

✓ + 1.5 pts Each cache line will contain multiple nodes of a binary tree. If a thread updates one of the node in a cache line, then it will lead to false sharing.

✓ + 1.5 pts Many levels (more no. of memory access)

+ 0 pts Incorrect

points arrangement

+ 1 pt 1

+ 0.5 pts 0.5

4.2 Part b

2 / 2 pts

✓ + 0.5 pts Parent nodes store bit vectors to track free entries in child nodes.

+ 1.5 pts not need to access all child node; find the child in parent node that have unallocated pid and go to only that child node

✓ + 1.5 pts Uses hardware instruction (bsf) for fast bit scanning, making operations efficient.

+ 0 pts Incorrect

4.3 Part c

5 / 5 pts

+ 0 pts Incorrect/Not Attempted

✓ + 1 pt TLB must be flushed whenever virtual to physical address mappings change

✓ + 1.5 pts Flushing is required when there is a context switch from a user process -> user process or kernel process -> user process

✓ + 1.5 pts Flushing is not required when switching from kernel process -> kernel process or user process -> kernel process

✓ + 1 pt This is because kernel virtual memory mappings are the same for all processes

Question 5

Question 5

6 / 10 pts

5.1 Part a

2 / 3 pts

- + 1.5 pts Only Single thread that calls fork is "forked" to create a new process
- + 1.5 pts Parent's entire virtual memory is copied (except the stack and execution context of other threads)
- + 0 pts Wrong/unrelated

🗨 + 2 pts Point adjustment

5.2 Part b

4 / 7 pts

- + 1.5 pts Fork Bomb(We will have 2^n processes)

✓ + 2 pts Before any write all processes will share the same physical memory pages

- + 0.5 pts When any process writes to these shared COW pages,
Page fault occurs and Reverse mapping is done to determine all processes sharing the page

✓ + 2 pts Kernel copies page and assigns new physical frame to writing proces, However other processes continue to share same old physical frame

- + 1 pt If Memory amplification exceeds physical RAM and swap space then the system will crash

- + 0 pts Wrong/Unrelated

Question 6

Question 6

7 / 10 pts

6.1 Part a

4 / 5 pts

- + 1 pt When the function returns, it stores data in stack to pre designated memory.

✓ + 1 pt It calls the exit system call and enters the ZOMBIE state.

✓ + 1 pt The SIGCHLD signal is sent to the parent.

✓ + 1 pt The parent calls the wait system call, reads the return value from the pre-designated memory location, and collects the exit status of the child.

✓ + 1 pt Finally, the child's state is completely erased from the OS

- + 0 pts [Click here to replace this description.](#)

6.2 Part b

3 / 5 pts

✓ + 3 pts Preventing Starvation

- + 2 pts Handling softirq requests from kernel threads

- + 0 pts Incorrect

- + 0 pts Not Attempted

Operating Systems Midterm Exam

2024-25 Sem II

Name: Sarthak Panda

Entry number: 2022 CS51217

- ☒ COL 331
- ☐ COL 633
- ☐ ELL 405
- ☐ ELL 783

Total: 60 marks

- Each question is for 10 marks.
- Prove and justify all your answers.

1. Answer the following questions:

- a. Consider a thread running on a core. It does not make any system calls, nor does it suffer from any exceptions. Will it continue to run forever without giving other threads a chance, if it has an infinite loop? [2]

No, ~~each~~ each CPU periodically gets timer interrupt from APIC hardware chip. These timer interrupts invoke the Scheduler which checks if the current process has exceeded its time limit quota, if yes reschedules a new thread.

- b. Consider a modern tickless or jiffy-less kernel. It does not use the solution that you proposed in 1.a. Additionally, there is no periodic event source. We still want the system to be responsive and have frequent context switches. No thread should monopolize a core. How will this scheme work? What are the advantages of such a scheme? [4]

To still have responsive system, we can put a limit on the number of instruction that a thread can execute before getting context switched out (i.e. after 1000 instructions [PC → PC+1000] (say) we will jump to kernel mode ~~and~~ Scheduler can check priority & fairness to allocate a new thread)

The advantage is the power consumption of jiffy less kernel will be lower as a clock/timer do not need to tick constantly. It will make it power efficient.

c. Now, consider a design where there are 8 cores. 7 of the 8 cores are not connected to any kind of timer. Only one of the cores can access any device that it wants. How can it coordinate the execution of the entire system? We clearly do not want any thread to run forever regardless of the core and monopolize it. [HINT: This will require concepts from Chapter 4] [4]

IPIs (Inter Processor Interrupts) can help us out in such a setup. Whenever the ~~one~~ Particular Core connected to timer (hence getting timer interrupt periodically) gets a timer interrupt, we can set the interrupt handler to call a Scheduler for current core & send IPIs to remaining 7 cores. When the other core receive an interrupt it draws the attention of kernel. Once each of these 7 cores are in kernel mode Scheduler can run and schedule some other thread, this ensure none of core gets monopolized.

2. Consider DLLs or shared objects. We wish to create a hot-swappable DLL, which can be replaced at run time. This means that a multi-threaded process can simply decide to upgrade its DLL while it is running. Any thread can issue a request to update the DLL. At that point of time, other threads could be executing functions in the old version. Regardless of this, the update process should happen seamlessly and cause minimal disruption.

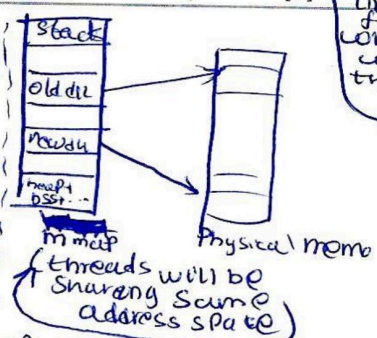
- a. How do we ensure that the disruption is minimized? Note that there will be a short period of time in which both the DLLs are active (new and old). [3]

When a new thread requests an update for dll, we will clear its Jump table ^{first} ~~start that~~ store the library path. So when this thread ¹ hits any library function it will call the 'dlopen' which will look for updated library (in current directory, /usr/lib...) After which Jump table for the current ~~thread~~ will be populated and new mapping will be setup from VM to Physical pages. The other threads also will clear Jump table ~~when~~ ^{when}

- b. When can the old DLL's code and data be unmapped? (and how?) [2]

In both we assume threads ~~share~~ keep their Jump table in TLS regions

Since we have updated the Jump table so whenever we hit library function (say like printf) we go to new dll in VA and get mapped to correct physical memory but to clear old dll in ~~any of thread mapping~~ we can clear them just before last thread clearing its old Jump table. this is enough because

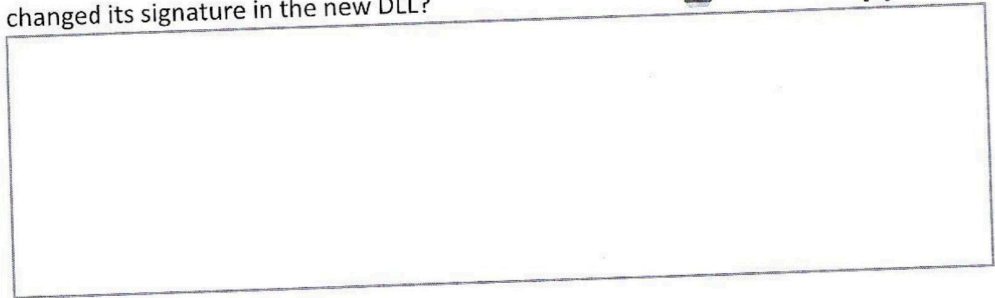


do not use library function which will update their dll too

- c. How can resources acquired by the old DLL be transferred to the new DLL? This includes open files, mapped virtual memory regions, network connections, etc. [Hint: Think about the `task_struct`] [2]

If we fork the old ~~task~~ task struct and then use `execv` to upload our new image into it then we get the new dll's as well as ^{task_struct} all resources acquired by old dll are automatically transferred by fork mechanism.

- d. How can a system be created to use the older version of a function, if it has changed its signature in the new DLL? [3]



3. Answer the following questions:

- a. Do all the threads use exactly the same virtual memory space? Justify your answer. [3]

Yes all the threads do share the same virtual memory space this because threads share heap, ~~code~~ text, data & bss. Only thing they do not share is stack & TLS to make thread context switch efficient we make them to use same virtual memory space.

- b. If they use the same virtual memory space, how are the stacks arranged in this shared space? [3]

The inspiration for arranging stacks was taken from kernel stack, where we preallocate a sufficiently large chunk of memory to a particular thread this makes stacks non-overlapping & as mentioned boosts up the thread context switch efficiency.

stack for thread 1	stack for thread 2	-	stack for thread n	heap	bss	data	text
--------------------	--------------------	---	--------------------	------	-----	------	------

- c. Design a mechanism to stop one thread from accessing the stack of another thread if all the threads share the same virtual address space. Can segmentation be somehow used? Justify your answer. [4]

Although we fixed the size of stack in ~~pre~~ but another thing left is: how do we get the starting pointers of the stacks for each thread. The stack segment ~~for~~ ~~current~~ CPU hold the ~~stack limit~~ stack-limit-size. So whenever thread with thread-ID "t" pointer to top of stack can be obtained by $t * \text{stack-limit-size} + \text{ESP}$;
 $\rightarrow$ from "SS" segment register
 (we assume there way to distribute thread IDs during creation of such threads.)

4. Answer the following questions.

a. What is wrong with a binary tree (from an OS perspective)? List at least two reasons. Focus on concurrency aspects also. [3]

i) In binary tree Since each node will be having atmost 2 children, the depth of resulting tree for huge amount of data is going to be very high. Due to this we will need to store too many Parent to child Pointer increasing overhead.
 ii) Since each node contains only one data hence a cacheline will end up storing many unrelated nodes. So then if two threads look on different parts still we will get false sharing.

b. What is the advantage that one gets if a parent node stores information about its children in an IDR tree? [2]

An Linux IDR tree is a 64-way tree and each node stores a 64 bit vector where i th bit indicating if i th subtree is free or not. Now to get left most '1' bit in this vector there is an hardware optimized instruction 'bsf' in x86 which makes this job of looking leftmost free Subtree very efficient and we would not need to go till leaf node.

c. In which case is it necessary to flush the TLB and reload the page table? And why? Why is it not necessary in the other cases? [5]

In Process Context Switch there is mainly three types of Switches
 i) Kernel $\rightarrow$ Kernel ii) user $\rightarrow$ Kernel iii) Kernel $\rightarrow$ user
 We only need to flush TLB and reload Page table (ie changing CR3 register entry) in ii) & b) ie when different user process is coming in when switching from Kernel to user. This is because TLB & page table Virtual Address to Physical Address and Virtual addresses differ from one user process to the other.
 But this is not required in Kernel $\rightarrow$ Kernel Switch because all the kernel threads share the same virtual address space and in case of user to kernel switch this is not needed since same user thread is getting promoted to kernel thread.
 We can check same user process in incoming or not using active-mm field of previous kernel task & mm of current incoming task if same then same user process.

to discom if it was free or not

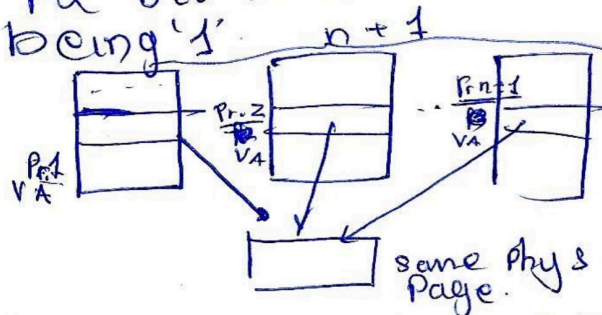
5. Answer the following questions:

a. What happens when a thread in a thread group is forked? [3]

When a thread is forked then another copy of same thread/task is created. The resources like open files, networks etc are directly inherited from parent process. The Page table of parent process is copied. Instead of copying the entire memory pointed by address space making it efficient.

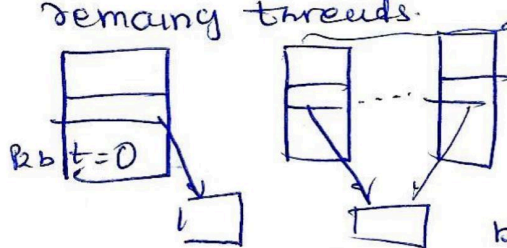
b. If a process repeatedly calls fork n times, what happens? [7]
Focus on the COW memory regions, actions taken when there is a write by a process and the potential to destabilize the system.

When we call fork " n " times there are $n+1$ processes all of them having same Page table and P2 bit in each of them's Page table being '1'.



Assume all are committed to write to same address for discussion

Now when any one of the forked process writes then a copy of page is made and P2 bit for that thread's Page table is set to '0' while it remains '1' for the remaining threads.



The only thing we need to carefully handle is when last 2 threads are remaining then when one of them writes P2 bit for both need to be set to Zero simultaneously or there will situation shown

which will lead to repeated copy of page.



To know how many threads are pointing to that page table from that thread group we can use an inverse page table.

6. Answer the following questions:

- a. Consider a work item that comprises a function pointer and its arguments. We wish to execute this item on a separate thread and later read its return value from a pre-designated memory location. What exactly happens when the function returns? How is the return value written to a pre-designated memory location, and how does the thread terminate? [5]

When a function returns it calls a `exit()` system call which enables the 'OS' to know that it has terminated (since ^{let it} hardware is dumb to know if it was completion of function so we must inform the OS). After this parent thread is sent an `SIGCHLD` signal which uses `wait()` system call & collect the return status and write to an MSR if needed. After which the Zombie thread in Task-Zombie state can be cleared off.

- b. What is the rationale for running some *softirq* threads in process context? [5]

To understand this, let us start from point where Hard IRQ (top half finishes) after which it schedules bottom half on *softirq* thread in *softirq* context. The problem comes when this *softirq* task schedules another *softirq* task and child *softirq* task schedules grandchild one & soon. So we must put a limit on number of iteration this can be done else these threads will be misused to monopolize high priority the CPU and can affect fairness. But now if bottom half task is not small & larger somewhat we still need to execute it to completion the *ksoftirqd* (kernel daemon) comes into play which runs periodically and checks if there is some ideal *softirq* it runs them in process context.

----- Rough sheet -----